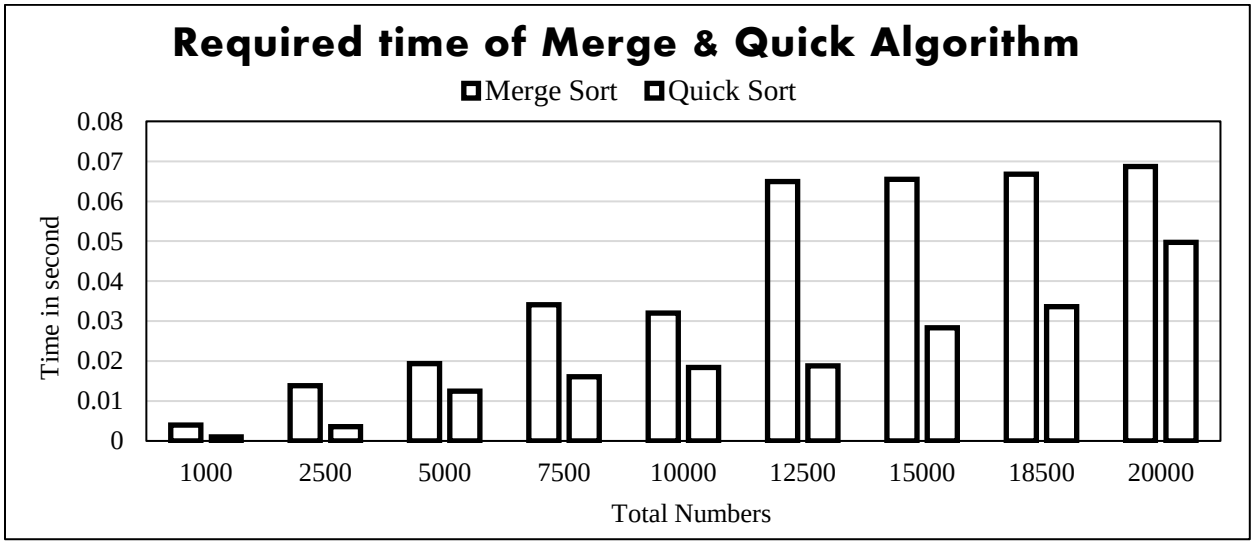
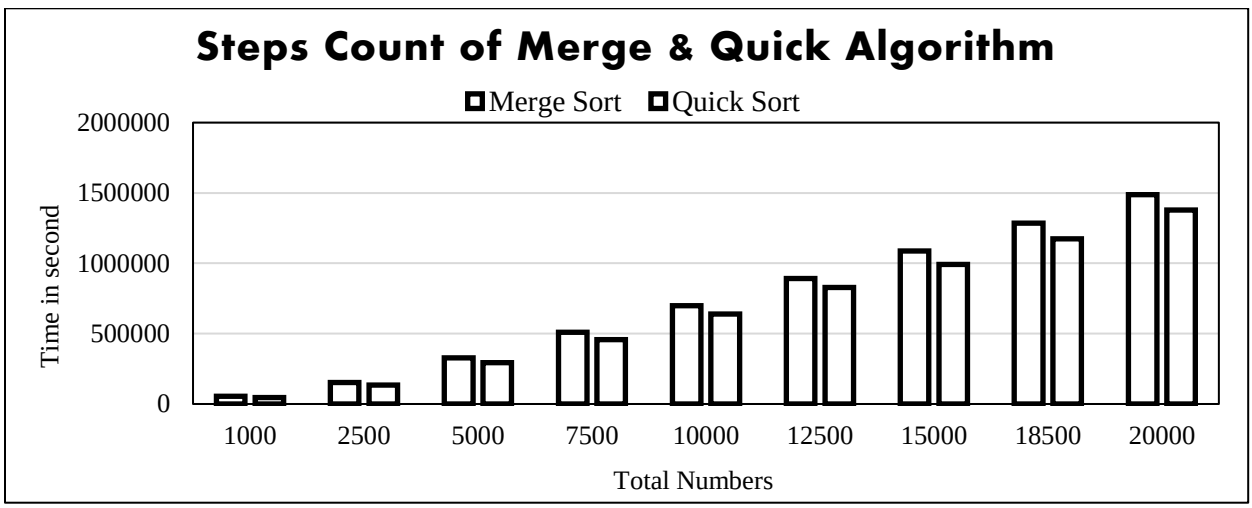
Output: (Steps)

Required Time				
	Merge Sort		Quick Sort	
No. of data	**Steps**	**Time**	**Steps**	**Time**
1000	54827	0.004s	46068	0.001s
2500	152259	0.01385s	134798	0.003588s
5000	327027	0.019382s	294535	0.012499s
7500	509599	0.034053s	458251	0.016076s
10000	699063	0.032023s	639004	0.018377s
12500	892583	0.064962s	828948	0.018818s
15000	1086707	0.065524s	991448	0.028292s
18500	1285219	0.066798s	1172937	0.033573s
20000	1488135	0.068672s	1377937	0.049733s

- *Graphical Interpretation:*





❖ Implemented Code (C++) for Merge & Quick Sort:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
using vec_1D=vector<ll>;
#define newl cout<<endl
ll step=0;
```

```
vec_1D Merge_Sort(vec_1D arr)
{
   int L=arr.size();
   if(L==1)
   {
      step++;
      return arr;
   }
   int n=L/2; step++;
   vec_1D left,right;
   step++;
   for(int i=0;i<n;i++)
   {
      left.push_back(arr[i]);step+=2;
   }
   step++;
   step++;
   for(int i=n;i<L;i++)
   {
      right.push_back(arr[i]);step+=2;
   }
   step++;
   left=Merge_Sort(left);step++;
   right=Merge_Sort(right);step++;
   vec_1D merge;
   int l=0,r=0;
   step++;
   for(int i=0;i<L;i++)
   {
      step++;
      if(r==int(right.size()) || left[l]<=right[r])
      {
         merge.push_back(left[l]);step+=2;
         l++;
         continue;
      }
      merge.push_back(right[r]);step++;
      r++;
   }
   step++;
   return merge;
}
```

```
vec_1D Quick_Sort(vec_1D arr)
{
   int L=0,h=arr.size();
   if (h<=1)
   {
      step++;
      return arr;
   }
   int p=(L+h)/2;step++;
   vec_1D left,right;
   step++;
   for(int i=0;i<h;i++)
   {
      step++;
      if(arr[i]<=arr[p]&&i!=p)
      {
         left.push_back(arr[i]);step+=2;
      }
      else if(i!=p)
      {
         right.push_back(arr[i]);step+=2;
      }
   }
   step++;
   left=Quick_Sort(left);step++;
   right=Quick_Sort(right);step++;
   vec_1D quick;
   step++;
   for(int i=0;i<int(left.size());i++)
   {
      step++;
      quick.push_back(left[i]);step++;
   }
   step++;
   quick.push_back(arr[p]);step++;
   step++;
   for(int i=0;i<int(right.size());i++)
   {
      step++;
      quick.push_back(right[i]);step++;
   }
   step++;
```

```
return quick;
}
```

```
int main()
{
   vec_1D list={1000,2500,5000,7500,10000,12500,15000,17500,20000};
   ofstream op;
   op.open("output.txt");
   op<<"Total Number\tMerge Sort Steps\tQuick Sort Steps\n";
   for(int i=0;i<9;i++)
   {
      ifstream ip;
      ip.open("input.txt");
      vec_1D merge(list[i]),quick(list[i]);
      for(int j=0;j<list[i];j++)
      {
         ip>>merge[j];
         quick[j]=merge[j];
      }
      ip.close();
      op<<"  "<<list[i]<<"\t\t";
      merge=Merge_Sort(merge);
      op<<"\t\t\t"<<step<<"\t";
      step=0;
      quick=Quick_Sort(quick);
      op<<"\t\t\t"<<step<<"\t\n";
      step=0;
   }
   op<<"\n";
}
```

❖ Implemented Code (C++) for MAX_MIN Algorithm:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
using vec_1D=vector<ll>;
#define newl cout<<endl
ll step=0;
vec_1D Max_Min(int a,int b,vec_1D A)
{
   if (b-a<=1)
   {
      step++;
      return {max(A[a],A[b]),min(A[a],A[b])};
   }
   else
   {
      vec_1D mx1_mn1=Max_Min(a,(a+b)/2,A);step++;
      vec_1D mx2_mn2=Max_Min((a+b)/2+1,b,A);step++;
      step++;
      return {max(mx1_mn1[0],mx2_mn2[0]),min(mx1_mn1[1],mx2_mn2[1])};
   }
}
vec_1D Brute_force_max_min(vec_1D A)
{
   vec_1D ans={A[0],A[0]};step++;
   step++;
   for (int i=0;i<int(A.size());i++)
   {
      if (A[i]>ans[0])
      {
         ans[0]=A[i];step+=2;
      }
      if (A[i]<ans[1])
      {
         ans[1]=A[i];step+=2;
      }
      step++;
   }
   step++;
   return ans;
}
int main()
{
   vec_1D list={1000,2500,5000,7500,10000,12500,15000,17500,20000};
   cout<<"Recursive Form";newl;
   for(int i=0;i<9;i++)
   {
      ifstream ip;
      ip.open("input.txt");
```

```
        vec_1D data(list[i]);
        for(int j=0;j<list[i];j++)
        {
           ip>>data[j];
        }
        ip.close();
        vec_1D mx_mn=Max_Min(0,list[i]-1,data);
        cout<<mx_mn[0]<<" "<<mx_mn[1]<<" "<<step;newl;
        step=0;
    }
    newl;
    cout<<"Brute Force Form";newl;
    for(int i=0;i<9;i++)
    {
        ifstream ip;
        ip.open("input.txt");
        vec_1D data(list[i]);
        for(int j=0;j<list[i];j++)
        {
           ip>>data[j];
        }
        ip.close();
        vec_1D mx_mn=Brute_force_max_min(data);
        cout<<mx_mn[0]<<" "<<mx_mn[1]<<" "<<step;newl;
        step=0;
    }
}
```

Output: (Steps)

Required Steps		
	Recursive Form	**Brute Force Form**
No. of data	**Steps**	**Steps**
1000	2045	3001
2500	5901	7501
5000	11805	15001
7500	16381	22501
10000	23613	30001
12500	32765	37501
15000	35422	45001
18500	37229	52501
20000	47229	60001

- *Graphical Interpretation:*